

Does Language Change LLM Token Cost? A Cross-Linguistic Comparison of Token Efficiency Across Three Tokenizers

Elliott E Chong

Abstract

Large Language Models (LLMs) convert text into tokens, but messages with identical semantic meaning may require different token counts across languages. This research paper compares Modern English, Modern Standard Chinese, Classical Chinese, Classical Latin, and Classical Attic Greek. The investigation is carried out with 45 statements that represent everyday communication, instructions, and simple reasoning, semantically matched across all languages. These statements are then encoded with 3 different tokenizers: OpenAI o200k_base, Qwen 2.5, Mistral 7B. Token efficiency is measured by token count, characters per token, bytes per token, and English-relative savings. The results aim to reflect whether language affects token count, and whether rankings differ by tokenizer.

Introduction

At their core, LLMs are transformers that receive text as token sequences rather than as words, sentences, or meanings. These token sequences are of significance to everyday users as token count affects context-window use and, in token-priced systems, potential user cost. Interestingly, equal meaning often does not mean equal token count because tokenizers learn different subword inventories from unequal language data. A short written form may be fragmented into many tokens, while a longer form may match frequent vocabulary units.

Cross-linguistic token efficiency is affected largely by both language form and tokenizer design. For instance, Modern Standard Chinese and Classical Chinese use Chinese characters, but they differ in grammar, vocabulary, and tolerance for omitted meaning. Classical Chinese can express some propositions compactly because subjects, objects, tense, and logical links may be inferred from context. Meanwhile, Latin and Attic Greek can encode grammatical relations through inflection rather than separate function words. On the other hand, in English grammatical information is often determined by word order and separate words. These differences make the five varieties a useful comparison set.

It is worth noting that human communicative efficiency does not equate to LLM token efficiency. Coupé et al. (2019) show that for spoken languages it is possible to balance information density and speech rate, but their result does not predict tokenizer behaviour. This study instead asks how fixed machine tokenizers segment written strings.

Existing literature reveals that multilingual tokenization can produce unequal token lengths and unequal practical access to LLM services (Ahia et al., 2023; Petrov et al., 2023). However, the research gap appears as Classical Chinese, Classical Latin, and Classical Attic Greek are underexplored in controlled multilingual token-count studies. This paper looks to investigate whether apparent linguistic concision remains visible after translation fidelity and tokenizer choice are controlled.

Literature review

Tokenization and subword vocabularies

Despite the prior mentioned research gap, there is abundant research on tokenization. Existing literature finds that tokenizers divide text into recurring units drawn from a fixed vocabulary, and byte-pair encoding merges frequent symbol sequences into larger subword units. Rare words, uncommon scripts, and poorly represented strings often split into more tokens. According to Sennrich et al. (2016), subword BPE is a practical solution for open-vocabulary neural translation. This asserted that tokenization is therefore data-driven compression rather than a neutral measure of word count.

SentencePiece provides an important background model for understanding multilingual tokenization. It tokenizes raw text without need for language-specific segmentation. This supports languages where words are not normally separated by spaces, such as Chinese. However, one should consider that its language-independent procedure does not ensure equal efficiency across language varieties. Vocabulary content and training-corpus distribution are still determinants of which strings receive compact units. The technical basis for this type of tokenizer is provided by Kudo and Richardson (2018).

Language representation and tokenizer quality

It is crucial that a multilingual tokenizer distribute finite vocabulary capacity across varying scripts and languages. Languages represented frequently in training data (such as English) may receive longer reusable subwords, while uncommon and underrepresented languages are likely to be divided into shorter fragments. Rust et al. (2021) find that tokenizer quality and vocabulary representation are essential for multilingual model performance. Based on their findings, the tokenizer should be treated as a meaningful experimental variable rather than as a fixed background tool.

Token inequality across languages

Ahia et al. (2023) links token fragmentation to access and affordability concerns. Based on their research, commercial LLM token pricing can charge different effective amounts for semantically similar content in different languages. Meanwhile Petrov et al. (2023) show that token-length inequality starts even prior to model inference and can persist across tokenizers. These studies reject the assumption that one token represents equal amounts of meaning in all languages. A key issue is that existing work often compares contemporary languages on benchmark

data, rarely isolating Classical Chinese from Modern Standard Chinese. Classical Chinese is also rarely compared with inflected classical languages under identical translation protocol.

Tokenizer choice

Tokenizer choice can affect efficiency and training cost substantially (Ali et al. 2024). A language can seem efficient under one tokenizer but may not be under another. Hence this study makes it a point to predict a language $\times$ tokenizer interaction rather than a fixed universal ranking.

Research question and hypotheses

Research question: For semantically equivalent written content, does using a different language reduce LLM token usage, and does result vary by tokenizer?

Given existing studies done, this paper hypothesises:

1. Mean token count differs across language varieties after matched source items are paired.
2. Classical Chinese has the lowest mean token count among the five varieties.
3. Language rankings differ across o200k_base, Qwen, and Mistral tokenizers.

Methodology

Corpus and design

The corpus (see corpus.csv) consists of 45 short propositions. 15 of them are everyday statements describing ordinary actions, possession, movement, time, and observation. 15 are instructions - commands, prohibitions, sequences, and conditional actions. The last 15 are reasoning statements expressing cause, contrast, condition, comparison, and simple inference. This dataset deliberately avoids idioms, specialist technical terms, and culture-specific references. Each proposition has been translated into Modern English, Modern Standard Chinese, Classical Chinese, Classical Latin, and Classical Attic Greek. Each language version receives token counts from three tokenizers, thus the final main dataset size is 675 observations: 45 items $\times$ 5 languages $\times$ 3 tokenizers.

Orthography and normalization

Before measurement all strings are normalized to Unicode NFC. For the sake of consistency both Chinese varieties use one fixed character standard, either simplified or traditional. Attic Greek uses one fixed polytonic spelling convention while Latin uses one fixed convention for u/v, i/j, capitalisation, and macrons. All languages follow the same punctuation policy, reducing punctuation-driven token differences.

Tokenizers

The 3 tokenizers used as mentioned above are

- OpenAI tokenizer: o200k_base
- Qwen tokenizer: Qwen2.5
- Mistral tokenizer: Mistral7B

Every tokenizer's model identifier, revision, library version, and special-token settings are recorded. This experiment only measures static text strings.

The tokenizer libraries are integrated into a single tokenizers.py script (see repo). It returns the following:

- Character count measures Unicode code points after NFC normalization.
- UTF-8 byte count measures encoded storage length.
- Token count measures tokenizer sequence length.
- Characters per token show average surface-character compression.
- Bytes per token show efficiency while accounting for script encoding differences.

Using the metrics obtained from the Python script, we are able to find:

English-relative token ratio compares each translation to its paired English item.

$$\text{Token Ratio} = \text{TokenLanguage} / \text{TokenEnglish}$$

Percentage token saving expresses the same comparison more intuitively.

$$\text{Saving} = 100 * (1 - \text{Token Ratio})$$

Positive saving means fewer tokens than English. Negative saving means more tokens than English.

Results

The table below showcases the mean and median tokens, characters and bytes for each language-tokenizer variation.

Language	Tokenizer	Mean tokens	Median tokens	Mean chars/token	Median chars/token	Mean bytes/token	Median bytes/token
Modern English	OpenAI o200k_base	8.87	8.00	4.66	4.64	4.66	4.64
Modern English	Qwen 2.5	8.89	8.00	4.64	4.64	4.64	4.64
Modern English	Mistral 7B	9.04	8.00	4.57	4.50	4.57	4.50
Modern Standard Chinese	OpenAI o200k_base	11.76	11.00	1.04	1.00	2.94	2.85
Modern Standard Chinese	Qwen 2.5	9.44	9.00	1.29	1.25	3.65	3.54
Modern Standard Chinese	Mistral 7B	16.58	15.00	0.74	0.77	2.09	2.15
Classical Chinese	OpenAI o200k_base	8.09	8.00	0.88	0.88	2.38	2.38
Classical Chinese	Qwen 2.5	7.18	7.00	0.99	1.00	2.67	2.67
Classical Chinese	Mistral 7B	11.40	11.00	0.64	0.62	1.73	1.69
Classical Latin	OpenAI o200k_base	12.33	11.00	3.29	3.18	3.29	3.18
Classical Latin	Qwen 2.5	13.47	13.00	2.99	2.94	2.99	2.94
Classical Latin	Mistral 7B	14.20	13.00	2.84	2.88	2.84	2.88
Classical Attic Greek	OpenAI o200k_base	31.20	28.00	1.41	1.43	2.79	2.82
Classical Attic Greek	Qwen 2.5	42.18	41.00	1.04	1.05	2.05	2.06
Classical Attic Greek	Mistral 7B	49.27	45.00	0.89	0.89	1.76	1.76

The data is further represented through Figures 1 and 2.

Figure 1 uses faceted paired-dot or box plots of English-relative token ratios. Each panel represents one tokenizer.

Figure 2 shows language ranking or mean token ratio across tokenizers.

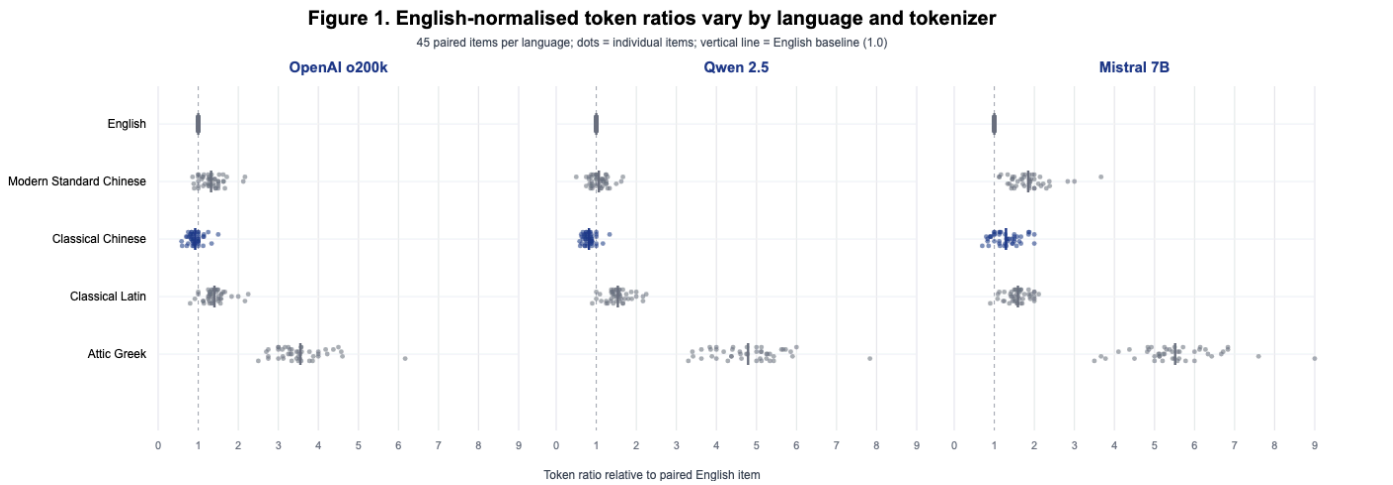


Figure 2. Token-efficiency rank changes across tokenizer families

Rank 1 = fewest mean tokens; cell labels show rank and mean English-relative token ratio

	OpenAI o200k	Qwen 2.5	Mistral 7B
English	#2 1.00×	#2 1.00×	#1 1.00×
Modern Standard Chinese	#3 1.32×	#3 1.07×	#4 1.85×
Classical Chinese	#1 0.92×	#1 0.82×	#2 1.29×
Classical Latin	#4 1.41×	#4 1.54×	#3 1.59×
Attic Greek	#5 3.55×	#5 4.79×	#5 5.51×

Source: 45 paired corpus items. Ratios are averages of item-level target-language / English token ratios.

Addressing the hypotheses

1. Figure 1 affirms that token counts differ significantly across languages.
2. Figure 2 shows that Classical Chinese has the lowest mean token count for o200k_base and Qwen 2.5 tokenizers, but not for Mistral 7B
3. Language rankings do differ between Mistral 7B and the other 2 tokenizers. o200k_base and Qwen 2.5 share the same ranking of Classical Chinese < English < Modern Chinese < Latin < Attic Greek while Mistral tokenizer has a ranking of English < Classical Chinese < Latin < Modern Chinese < Attic Greek.

Discussion and limitations

Our findings have uncovered the following:

1. Token count differences arise from interaction between linguistic form and tokenizer vocabulary.
2. Fewer written characters do not automatically produce fewer tokens, and more bytes do not automatically produce more tokens.
3. A tokenizer may encode frequent Chinese-character sequences efficiently while fragmenting Greek or Latin morphology, while another tokenizer may produce a different result because its vocabulary was trained differently.

Our second hypothesis that Classical Chinese would have the lowest mean token count receives only conditional and not universal support since this is not the case under some tokenizers.

Since language rankings differ across tokenizers, our third hypothesis supports the stand that language efficiency cannot be separated from tokenizer choice.

It is crucial that the limitations of this study are addressed:

1. The study does not measure internal reasoning, embeddings, attention, model accuracy, latency, energy, or exact computation.
2. Token count is a sequence-length and acts only as a possible proxy for cost.
3. The corpus is small and cannot represent all registers, genres, or historical variation.
4. A downside of using an English anchor is that it may introduce source-language bias.
5. Tokenizer versions may change over time or become obsolete

Conclusion

To conclude this study we can assert that semantically matched text can require different token counts across language varieties, and that these differences should be interpreted as properties of language–tokenizer combinations. While Classical Chinese may show low token counts, its ranking may vary based on tokenizer and translation condition.

The final, defensible conclusion is thus conditional: token efficiency is dependent on tokenizer version, content of the corpus, translation fidelity, orthographic policy, and language variety.

Core bibliography

- Ahia, O., et al. (2023). *Do All Languages Cost the Same? Tokenization in the Era of Commercial Language Models*. EMNLP, 9904–9923. [Paper](#)
- Ali, M., et al. (2024). *Tokenizer Choice for LLM Training: Negligible or Crucial?* Findings of NAACL, 3907–3924. [Paper](#)
- Coupé, C., Oh, Y. M., Dediu, D., & Pellegrino, F. (2019). Different languages, similar encoding efficiency. *Science Advances*, 5(9), eaaw2594. [Paper](#)
- Kudo, T., & Richardson, J. (2018). *SentencePiece*. EMNLP System Demonstrations, 66–71. [Paper](#)
- Petrov, A., La Malfa, E., Torr, P. H. S., & Bibi, A. (2023). *Language Model Tokenizers Introduce Unfairness Between Languages*. NeurIPS. [Paper](#)
- Rust, P., Pfeiffer, J., Vulić, I., Ruder, S., & Gurevych, I. (2021). *How Good Is Your Tokenizer?* ACL-IJCNLP, 3118–3135. [Paper](#)
- Sennrich, R., Haddow, B., & Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units*. ACL, 1715–1725. [Paper](#)

- Yao, X., Wang, M., Chen, B., & Zhao, X. (2025). *WenyanGPT: A Large Language Model for Classical Chinese Tasks*. IJCAI-25. [Paper](#)